Copiloto de Posventa Autónomo y Gestión Inteligente de Devoluciones (ReFlow)

1. Planteamiento del Problema y Arquitectura Técnica

El problema de negocio

Las devoluciones y consultas de estado de pedido (“¿Dónde está mi paquete?”) representan más del 60% del volumen de soporte en e-commerce. Validar si un producto cumple la política de devolución y tramitar la etiqueta de envío consume un tiempo excesivo del equipo humano de atención al cliente.

Arquitectura Técnica del Sistema (Return Workflow)

La arquitectura de la solución se diseña bajo un enfoque modular, separando la persistencia de datos, la orquestación de la inteligencia artificial y la interfaz de usuario. El sistema se compone de los siguientes bloques principales:

1. Servidores MCP (Model Context Protocol)

Actúan como la capa de herramientas e integración externa, permitiendo a los LLMs interactuar con sistemas de forma segura y estandarizada:

- **MCP Supabase (mcp_supabase.py):** Servidor unificado de datos y conocimiento. Gestiona tanto el acceso en tiempo real a la base de datos relacional (historial de pedidos, usuarios, estados en PostgreSQL) como la búsqueda semántica vectorial (RAG sobre políticas de devolución usando pgvector).
- **MCP Logistics (mcp_logistics.py):** Servidor de acciones externas. Actúa como un conector simulado para interactuar con agencias de transporte, emitir etiquetas PDF de devolución y simular el envío de correos electrónicos transaccionales al cliente.

2. Sistema Multi-Agente (LangChain)

El núcleo de razonamiento se implementa como una cadena secuencial de agentes construida con **LangChain**, dividida en tres agentes especializados para evitar la saturación del contexto del LLM y garantizar la seguridad:

- **Agente de Triaje y Empatía:** Primer punto de contacto. Atiende al usuario, comprende el motivo de su insatisfacción y extrae parámetros clave (como el order_id) de forma conversacional.
- **Agente Evaluador de Políticas:** Actúa como auditor. Consulta el historial del pedido (SQL) y la base vectorial de normas (RAG) para verificar de forma estricta si la devolución es elegible basándose en plazos, categoría del producto y estado.
- **Agente de Resolución:** Si la devolución es elegible, invoca las herramientas logísticas para generar la etiqueta de transporte, registra la devolución en la base de datos y redacta el mensaje final. Incluye una condición de **Human-in-the-Loop (HITL)** que pausa el proceso automático si el importe a devolver supera los 100€, derivando el caso a un administrador para su aprobación manual.

3. Backend (FastAPI) y Frontend (Streamlit)

- **Backend REST y Streaming (FastAPI):** Expone los flujos de LangChain como servicios web seguros. Gestiona las sesiones de chat, la persistencia del estado en Supabase y emite en tiempo real las respuestas generadas por los agentes.
- **Interfaces de Usuario (Streamlit):** Vista Cliente — chat interactivo donde el usuario final puede tramitar su devolución de forma conversacional y descargar su etiqueta de envío. Vista Admin — panel de

control (dashboard) para la supervisión humana, que permite revisar las devoluciones de alto valor pausadas por el sistema y aprobarlas o rechazarlas con un solo clic, reanudando la ejecución del agente.

Modelo de base de datos

Entidad	Descripción
customers	Clientes registrados con datos de contacto y dirección de envío.
products	Catálogo de productos con SKU, categoría, precio y peso.
orders	Pedidos realizados por un cliente con su estado y fecha de entrega.
order_items	Líneas de detalle de cada pedido (producto, cantidad, precio).
return_requests	Solicitudes de devolución con motivo, estado y resolución.
return_items	Productos específicos incluidos en una solicitud de devolución.
return_policies	Políticas de devolución por categoría (plazos, condiciones, tasas).
policy_embeddings	Chunks vectorizados de las políticas para búsqueda semántica (RAG).
chat_sessions	Sesiones de conversación entre un cliente y el asistente.
chat_messages	Mensajes individuales (user/assistant/system) dentro de una sesión.

El esquema completo (tipos de columna y claves) se define en las migraciones SQL de `supabase/migrations/` y en el diagrama `supabase/ER_diagram.md`.

Tabla de Herramientas (Tools) por Servidor MCP

Servidor MCP	Herramienta	Entradas	Descripción y agente asignado
mcp_supabase.py (Datos y Políticas)	get_order_details	order_id	Agentes 1 y 2: consulta en PostgreSQL los detalles del pedido, fechas y productos comprados.
	get_customer_return_history	customer_id	Agente 2: recupera el historial de devoluciones del cliente para detectar posibles abusos.
	search_return_policy	query, category	Agente 2: realiza una búsqueda semántica RAG usando pgvector sobre la normativa de la tienda.
	create_return_record	order_id, customer_id, reason...	Agente 3: inserta una solicitud de devolución en Supabase.
	update_return_status	return_id, status	Admin / HITL: actualiza el estado de la devolución desde el panel de control manual.
mcp_logistics.py (Logística)	generate_shipping_label	order_id, items	Agente 3: genera un PDF simulado de la etiqueta de transporte con reportlab.
	send_customer_notification	email, message...	Agente 3: simula el envío de un correo electrónico transaccional con el enlace al PDF generado.

2. El Flujo Completo Paso a Paso

- El usuario entra en Streamlit: escribe “Hola, quiero devolver el pedido #12345”.

- Streamlit hace una llamada HTTP POST: envía ese texto a http://localhost:8000/api/chat en formato JSON.
- FastAPI recibe el JSON: valida que el texto y el order_id tengan el formato correcto mediante esquemas de Pydantic.
- FastAPI despierta a LangChain: transfiere los datos a la cadena de agentes en ejecución.
- Los agentes trabajan en secuencia: el Agente de Triaje extrae el identificador #12345; el Agente Evaluador llama al MCP Supabase (SQL) para comprobar la fecha de compra y al MCP Supabase (RAG/pgvector) para comprobar si el producto admite devolución; el Agente de Resolución llama al MCP Logistics para crear la etiqueta de envío.
- LangChain devuelve el resultado: notifica la salida final a FastAPI.
- FastAPI responde a Streamlit: envía la estructura JSON final {"status": "approved", "response": "Tu devolución ha sido aprobada...", "label_url": "http://.../etiqueta.pdf"}.
- Streamlit renderiza el resultado: muestra el mensaje en la pantalla del usuario e incrusta el botón de descarga del PDF.

Resumen del rol de FastAPI: funciona como la capa de integración limpia, asíncrona y segura entre la interfaz de usuario y el motor multi-agente subyacente.

3. Organización del Backlog (Épicas e Historias de Usuario)

Épica 1: Capa de Datos en Supabase y Servidores MCP (Semana 1) — 20 horas

Objetivo: configurar el proyecto en Supabase (PostgreSQL + pgvector) y exponer las herramientas a través del estándar MCP.

- US 1.1 Esquema relacional en Supabase (SQL): migraciones con las tablas customers, products, orders, order_items, return_requests, return_items.
- US 1.2 Base vectorial de políticas (pgvector): tabla policy_embeddings (vector(1536)) y función RPC match_policy_embeddings para búsqueda por similitud de coseno.
- US 1.3 Servidores MCP: mcp_supabase.py (get_order_details, get_customer_return_history, search_return_policy, create_return_record, update_return_status) y mcp_logistics.py (generate_shipping_label, send_customer_notification).

Épica 2: Sistema Multi-Agente en LangChain (Semanas 2 y 3) — 35 horas

Objetivo: construir el motor de razonamiento multi-agente, la gestión de estado y las reglas de negocio.

- US 2.1 Diseño del estado compartido: definir la estructura ReturnProcessState que se pasa entre agentes de la cadena.
- US 2.2 Agente 1 (Triaje y Empatía): prompt para saludar, entender el problema y extraer order_id e intenciones del usuario.
- US 2.3 Agente 2 (Evaluador de Elegibilidad): prompt para invocar las herramientas de mcp_supabase.py y lógica determinista de validación (ej. días <= 30 y policy_ok == True).
- US 2.4 Agente 3 (Resolución y Logística): prompt para invocar mcp_logistics.py y redactar el mensaje final de respuesta al cliente.
- US 2.5 Lógica Human-in-the-Loop (HITL): si refund_amount > 100€, marcar la devolución como “pendiente_aprobacion” y detener la respuesta automática hasta requerir aprobación manual.

Épica 3: Backend REST y Streaming en FastAPI (Semana 4) — 20 horas

Objetivo: exponer los flujos de agentes como servicios web asíncronos y seguros.

- US 3.1 Endpoints de chat: esquemas Pydantic v2 (ChatRequest, ChatResponse) y endpoint POST /api/chat/stream que emite en tiempo real el razonamiento de los agentes hacia la interfaz.
- US 3.2 Endpoints del panel de administración (HITL): GET /api/admin/pending-returns y POST /api/admin/approve-return, que dispara la ejecución del Agente de Resolución en LangChain.
- US 3.3 Persistencia de sesión: historial de conversación respaldado en la conexión PostgreSQL de Supabase (o en memoria local durante el desarrollo), indexado por session_id.

Épica 4: Frontend e Interfaz Admin en Streamlit (Semana 5) — 20 horas

- US 4.1 Vista del cliente (chat): componente de chat interactivo que consuma las respuestas de FastAPI; muestra el pedido e incrusta el botón de descarga del PDF cuando la devolución es aprobada.
- US 4.2 Vista de administración (dashboard y HITL): panel de métricas (aprobadas / rechazadas / escaladas) y tabla interactiva para aprobar o rechazar devoluciones pendientes (>100€).

Épica 5: Observabilidad, Evaluación y QA (Semana 6) — 15 horas

- US 5.1 Observabilidad: conectar LangSmith (o Langfuse como alternativa) u OpenTelemetry para auditar latencias, uso de herramientas MCP y consumo de tokens.
- US 5.2 Evaluación de calidad (AI as Judge / RAGAS): Golden Dataset con 15 consultas de prueba y métricas de Faithfulness y precisión en la recuperación vectorial con pgvector.

Épica 6: Redacción de la Memoria y Defensa (Semanas 7 y 8) — 30 horas

- US 6.1 Redacción de la memoria: introducción y objetivos; estado del arte (MCP, Supabase/pgvector, sistemas multi-agente); arquitectura técnica (FastAPI, LangChain, MCP, Supabase); evaluación y análisis de costes.
- US 6.2 Preparación de la defensa: presentación de diapositivas (15-20 min) y vídeo de respaldo con la demostración del flujo completo.

4. Cronograma Recomendado (Plan de 8 Semanas)

- Semana 1 — Épica 1: Supabase (PostgreSQL + pgvector) + servidores MCP (20h)
- Semana 2 — Épica 2: LangChain, agentes básicos (18h)
- Semana 3 — Épica 2: LangChain, reglas y Human-in-the-Loop (17h)
- Semana 4 — Épica 3: backend FastAPI + persistencia en Supabase (20h)
- Semana 5 — Épica 4: frontend Streamlit, cliente y admin (20h)
- Semana 6 — Épica 5: observabilidad y evaluación (15h)
- Semana 7 — Épica 6: redacción de la memoria (15h)
- Semana 8 — Épica 6: ajustes de memoria y ensayos de defensa (15h)

Consejos clave para no sobrepasar los tiempos

- Aplica la filosofía “Menos es Más” en la UI: los componentes nativos de st.chat_input, st.chat_message y st.dataframe son suficientes para un TFM impecable.
- Prioriza los servidores MCP en local: ejecútalos dentro del mismo entorno de Python usando Streamable HTTP local para evitar problemas de red durante la fase inicial de desarrollo.

- Escribe la memoria en paralelo: no esperes a la semana 7; documenta arquitectura y decisiones técnicas a medida que avanzas.